

MASTER PROMPT: Gas Station Management Web App

Paste this entire prompt into your coding agent (opencode + Gemini). Adjust names/branding as needed.

PROJECT OVERVIEW

Build a full-stack web application for managing a gas station's daily operations. The station works with two fuel vendors: **Al Mohit** and **Afriquia**. The app must let an admin manage workers, track charges/expenses, scan paper receipts with a device camera, automatically detect which vendor the receipt belongs to, extract the amount, and file it under the correct vendor ledger. The admin needs a dashboard with weekly/monthly/yearly views and the ability to export data as PDF and Excel.

TECH STACK (recommended, adjust if agent prefers otherwise)

- **Frontend:** React + Vite + TailwindCSS
- **Backend:** Node.js + Express (or Next.js API routes if using Next full-stack)
- **Database:** PostgreSQL (or SQLite for simple/local deployment) via Prisma ORM
- **Auth:** JWT-based sessions, bcrypt password hashing
- **OCR:** Tesseract.js (client or server side) OR Google Cloud Vision API / Gemini API for receipt text extraction (since you're using Gemini already, use the Gemini API multimodal vision endpoint to read the receipt image directly and return structured JSON — this is more reliable than raw OCR + regex)
- **PDF export:** pdfkit or puppeteer (render HTML to PDF)
- **Excel export:** exceljs or SheetJS (xlsx)
- **Camera capture:** `browser.getUserMedia / <input type="file" accept="image/*" capture="environment">` for mobile camera access

CORE DATA MODEL

User (admin/worker)

- id, name, email, password_hash, role (admin | worker), created_at

Worker

- id, name, position, phone, hire_date, active (bool)

Shift / WorkLog (optional but useful)

- id, worker_id, date, hours_worked, notes

Vendor

- id, name (enum: "Al Mohit" | "Afriquia")

Receipt

- id, image_url, vendor_id (nullable until classified), amount, currency

- extracted_raw_text, confidence_score

- status (pending_review | confirmed | rejected)

- scanned_by (user_id), scanned_at

Charge / Transaction

- id, vendor_id, receipt_id (nullable, manual entries allowed), amount

- category (fuel_purchase | maintenance | salary | other)

- date, description, created_by

MonthlySummary / cached aggregates (optional, can be computed on the fly)

FEATURE 1 — AUTHENTICATION & ROLES

- Login page (email + password)
- Roles: `admin` (full access) and `worker` (limited: can scan receipts, view own shifts, cannot see financial totals or manage other workers)
- JWT stored in httpOnly cookie
- Protect all admin routes with middleware checking role
- Password reset flow (optional nice-to-have)

FEATURE 2 — WORKER MANAGEMENT (Admin only)

- CRUD interface: add/edit/deactivate workers
- Assign login credentials to workers if they need app access
- Track basic info: name, position, phone, hire date, active status

- Optional: simple shift/attendance log

FEATURE 3 — RECEIPT SCANNING & AUTO-CLASSIFICATION

This is the core “smart” feature. Flow:

1. User taps “Scan Receipt” → opens camera (`capture="environment"` on mobile, or webcam on desktop)
2. Image is sent to backend
3. Backend calls **Gemini’s multimodal API** with a prompt like:

```
"You are reading a fuel station receipt. Identify: (1) the vendor name — it will be either 'Al Mohit' or 'Afriquia' or unknown, (2) the total amount, (3) the currency, (4) the date on the receipt. Look for logos, header text, and known keywords for each vendor [give the agent a list of exact strings/logos/keywords unique to each vendor's real receipts — you'll need to supply a few sample receipt images for both vendors so the agent/model can learn the visual/textual cues]. Return ONLY valid JSON: {  
  "vendor": "Al Mohit" | "Afriquia" | "unknown", "amount": number, "currency":  
  string, "date": "YYYY-MM-DD", "confidence": 0-1 } "
```

4. Backend parses the JSON response
5. If `confidence >= 0.8` and vendor is recognized → auto-file the charge under that vendor’s ledger, `status = confirmed`
6. If `confidence < 0.8` or `vendor = unknown` → save as `pending_review`, show it in an admin review queue with the image and extracted fields editable, admin manually confirms vendor + amount
7. Store the original receipt image (S3, local disk, or DB blob) for audit purposes

Important implementation note for the agent: since vendor detection depends on real receipt layouts, tell the agent explicitly: “I will provide sample receipt images from Al Mohit and Afriquia — use them to define the detection keywords/logo regions before finalizing the prompt to Gemini’s vision endpoint.” Supply a handful of real (redacted if needed) sample receipts during development so the classification prompt/rules can be tuned.

FEATURE 4 — CHARGES / EXPENSE TRACKING

- List view of all charges, filterable by vendor, date range, category
- Manual entry form (for cash purchases, non-receipt expenses, salaries, maintenance

costs)

- Edit/delete with admin permission
- Running totals per vendor, per category

FEATURE 5 — ADMIN DASHBOARD

- Summary cards: total spend this week / month / year, per vendor breakdown
- Charts (bar/line) showing spend trends — use Recharts or Chart.js
- Toggle between Weekly / Monthly / Yearly views (date range picker)
- Table of recent transactions with quick filters (vendor, category, worker who logged it)
- Pending receipt review queue (low-confidence scans needing manual confirmation)

FEATURE 6 — EXPORT (PDF & Excel)

- Export button on dashboard and on any filtered table view
- **Excel export:** use `exceljs`, generate a workbook with sheets per vendor + a summary sheet, respecting the currently selected date range (week/month/year)
- **PDF export:** render a clean report (station name, date range, vendor totals, itemized transactions, charts as images if feasible) using `puppeteer` (HTML→PDF) for best formatting control, or `pdfkit` for a lighter dependency
- Both exports should respect current filters (vendor, date range, category) so admin can export exactly what they're viewing

NON-FUNCTIONAL REQUIREMENTS

- Mobile-responsive (workers will likely scan receipts from a phone)
- Works offline-tolerant for scanning (queue upload if connection drops) — optional stretch goal
- Basic input validation and error handling everywhere
- Environment variables for API keys (Gemini API key, DB connection string, JWT secret) — never hardcode
- Seed script to create an initial admin account

SUGGESTED BUILD ORDER (give this to the agent as a step-by-step plan)

1. Scaffold project (frontend + backend), set up DB schema/migrations
2. Auth system (login, JWT, role middleware) + seed admin user
3. Worker CRUD (admin panel)
4. Vendor + Charge models with manual entry CRUD
5. Dashboard with basic totals and date-range filtering (no charts yet)
6. Add charts (Recharts/Chart.js)
7. Receipt upload + camera capture UI
8. Gemini vision integration for extraction/classification
9. Pending-review queue UI for low-confidence scans
10. PDF export
11. Excel export
12. Polish: responsive design, loading states, error handling
13. Testing with real sample receipts from both vendors, tune classification prompt

DELIVERABLES TO ASK THE AGENT FOR

- Full source code (frontend + backend) in a clean folder structure
- `.env.example` file listing all required environment variables
- README with setup/run instructions
- Database migration/seed scripts
- A short section in the README explaining how to adjust the vendor-detection prompt if a third vendor is added later

Instructions to the coding agent: Build this iteratively following the build order above. Ask me for sample receipt images from Al Mohit and Afriquia before finalizing the classification prompt. Confirm the tech stack choices with me before scaffolding if you'd prefer alternatives (e.g., Next.js instead of separate React+Express).

